



Add Support for RFC 5723 'Session Resumption' and RFC 6023 'Childless Initiation of SA'

Basic Information

Name and Contact Information

- **Name:** Nupur Agrawal
- **Email:** nupur202000@gmail.com
- **Github:** [murex971](https://github.com/murex971)
- **Phone Number:** +91 6377323544

University and Current Enrollment

- **University:** Indian Institute of Technology Roorkee
- **Major:** Applied Mathematics
- **Degree:** Integrated Masters of Science

Availability

- Reachable anytime easily through email or gitter.
- Available for video chat anytime between 10:00 am - 2:00 am IST during vacations and 6:00 pm - 2:00 am IST otherwise.

Background Information

About me

I am a 19-year-old, sophomore currently enrolled at IIT Roorkee. I developed a passion for development in my freshmen year and from then, most of my time goes into reading and writing software. I have been contributing to open-source regularly since my freshman year.

While exploring, I came across the field of networking, optimization techniques, and algorithms, packet analysis, etc. Since then I seek every opportunity to learn more about this. I try to solve some CTF questions and have been doing **binary reversing and pwnning**, using decompilers like *gdb* and *radare*.

I coded an SMTP (E-Mail) Server in C using Berkeley Sockets which has the ability to send and receive mails over LAN (using IP) as an assignment in my first year. That was my introduction to networking and ever since I tried to grow in the same.

I am part of [SDSLabs](#), a technical group of our campus, where I have worked in teams on various development projects, developed many new applications while maintaining several old ones. Most of the work is done on strict timelines with a lot of focus on test-driven development and following the best practices to structure the code as per the requirements of the task.

We developed and maintain [Backdoor](#), a platform for Computer Security Challenges. We also hold regular competitions on this platform.

As of coding style, I document my code and leave necessary comments wherever required.

I can code in Java, PHP, C++, C, Python, JavaScript, Solidity, experienced in tools like nmap, Wireshark, Burp suite and am proficient in using Git, VSCode, and Vim.

Related Experience

1. [GSoC 2019](#)

- I am an open-source enthusiast and always seek an opportunity to learn more by contributing. I applied for GSoC last year, as well, for a project under the organisation [phpMyAdmin](#).

- My previous year GSoC project majorly involved twig refactoring and UI enhancement.

2. Rubeus

- A cross-platform 2D game engine written in C++ for beginners.
- I worked on the messaging system of the engine and fixed some logical issues in the physics of the engine.
- Source code available at github.com/sdslabs/Rubeus

3. Rootex

- Rootex is a 3D multithreaded game engine that provides fast and high quality 3D rendering using DirectX, an embedded Lua scripting environment, support for multiple input hardware, and is planned to be the backbone of an unannounced project being developed at SDS Labs. Rootex Engine comes with its own editor called the Rootex Editor which is being developed using dear ImGui
- I spearheaded the development of its multi-threaded task manager and physics engine.
- Source code available at github.com/sdslabs/Rootex

4. Online Voting Machine

- A user-friendly web app to make voting hassle-free and secured using blockchain technology.
- It was made during a hackathon. I worked on the backend of the web app that involved Flask with Python.
- Source code available at github.com/murex971/OnlineVotingMachine

Description of the Project

The project involves work on adding support for **RFC 5723** primarily. RFC 5723 proposes an extension to IKEv2 (Internet Key Exchange v2) that allows a client to re-establish an IKE Security Association with a gateway in a highly efficient manner, utilizing a previously established IKE SA.

As this RFC might not be enough for the entire GSoC period, I will be working on further adding support for **RFC 6023**.

RFC 6023 describes an extension to the Internet Key Exchange version 2 (IKEv2) protocol that allows an IKEv2 Security Association to be created and authenticated without generating a Child SA.

If I am able to implement both RFC 5723 and RFC 6023 well before the GSoC timeline, I plan to stretch my goals by working on RFC 6290.

RFC 5723

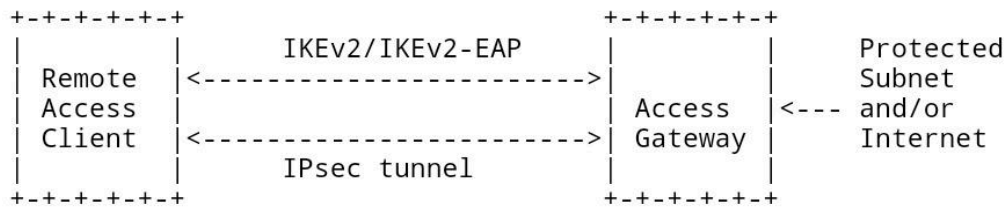
Problem or (and) Need

- To re-establish security associations (SAs) upon a failure recovery condition is time-consuming.
- Usability is also affected when the re-establishment of an IKE SA involves user interaction for re-authentication.

Solution

- The client stores the state about the previous IKE SA locally. The gateway has two options for maintaining the IKEv2 state about the previous IKE SA:
 - **Ticket by Reference**
 - **Ticket by Value**

(a)



(b)

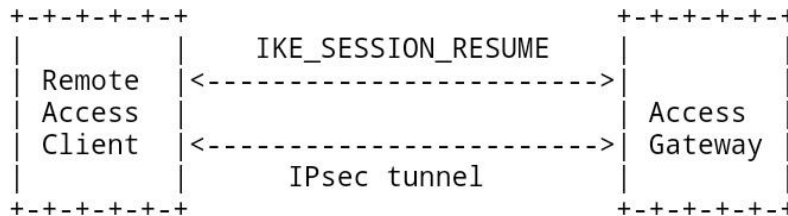


Figure 1: Resuming a Session with a Remote Access Gateway

Implementation in Libreswan

The following Notify Message types have been assigned by IANA.

Notification Name	Value
TICKET_LT_OPAQUE	16409
TICKET_REQUEST	16410
TICKET_ACK	16411
TICKET_NACK	16412
TICKET_OPAQUE	16413

Two new source code files will be created:

programs/pluto/ikev2_resume.{h,c} for new methods e.g. build payloads, new IKEv2 exchanges, etc.

programs/pluto/ikev2_resume.c will define the event for building **TICKET_LT_OPAQUE Notify Payload (1)** and **TICKET_OPAQUE Notify Payload (2)** structure of which are in following images from RFC 5723. The other notification payloads have no data and will be addressed in the

same files. The file also contains the function initiating **IKE_SESSION_RESUME** Exchange.

1)

```

      0               1               2               3
    0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
| Next Payload |C| Reserved | Payload Length |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
| Protocol ID  | SPI Size = 0 | Notify Message Type |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|                               Lifetime                               |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|                               Ticket                               |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+

```

2)

```

      0               1               2               3
    0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
| Next Payload |C| Reserved | Payload Length |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
| Protocol ID  | SPI Size = 0 | Notify Message Type |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|                               Ticket                               |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+

```

The **ticket struct** needs to be defined in **programs/pluto/packet.c** (**in_struct()** and **out_struct()**) by considering and discussing all the factors with the mentor regarding the Libreswan's need and integrity with it.

For reference, the **Appendix** in the RFC is already given.

Main Working

- After the basic functionalities and structures are well coded, we come to process the ticket for session resumption. It involves primarily 3 steps as briefed in the document.
 - **Requesting a Ticket**

A client may request a ticket in **IKE_AUTH** exchange, **CREATE_CHILD_SA** exchange, informational exchange or session resumption exchange. The request involves the **TICKET_REQUEST** notification payload.

Code changes and Impact: major changes in *programs/pluto/ikev2** to add the code for requesting ticket function using the mentioned payload.

■ **Receiving a Ticket**

When the client accepts the ticket, it stores it in its local storage for later use, along with the IKE SA to which the ticket refers. This process can be divided into two milestones or phases as the project will progress.

- ❑ In initial use case, the ticket is encrypted and contains all the information.
- ❑ In the second phase or milestone, ticket by reference could be implemented.

■ **Presenting a Ticket**

When the client wishes to recover from an interrupted session, it presents the ticket to resume the session. The resumption process consists of some preparations, an IKE_SESSION_RESUME exchange, an IKE_AUTH exchange and finalization.

Code changes and Impact: The functionalities for validating the presented ticket will be added in *programs/pluto/ikev2_resume.c*

○ Interaction and other side works

■ **Interacting with Session Resumption**

A client that wakes up from sleep could do either a **session resumption** or a **mobike resumption**. For this different scenarios would be considered that could commonly happen and suitable outcome need to be decided among various possibilities and combinations.

Different usage scenarios need to be taken into account like if a client temporarily loses network connectivity, if the connectivity problems affect a large number of clients, etc.

■ **Ticket Identity and Lifecycle**

The lifetime of the ticket sent by the gateway SHOULD be the minimum of the IKE SA lifetime (per the gateway's local policy) and its reauthentication time.

When credentials associated with the IKE SA are invalidated, the ticket **MUST** be deleted. When the IKE SA is rekeyed, the ticket is invalidated, and the client **SHOULD** request a new ticket.

Documentation and Testing

- After the implementation and testing of the extension proposed, the related commands documentation will be addressed in **programs/configs/d.ipsec.conf/<command/ msg_name>.xml**
- Testing of the proposed extension will be added to **testing/pluto** with appropriate changes to console outputs.

Minor Changes

Some minor code changes involve adding keywords and other naming in **include/, lib/libipsecconfig/keywords.c**, etc.

RFC 6023

After the implementation of RFC 5723, I am planning to work on RFC 6023 which describes an extension to the Internet Key Exchange version 2 (IKEv2) protocol that allows an IKEv2 Security Association(SA) to be created and authenticated without generating a Child SA.

Remark: An IKEv2 SA without any Child SA is not a fruitless endeavor.

● Process

- A normal IKE_SA_INIT exchange from the initiator.
- The responder must include the Notify payload (as given in the following image) within the IKE_SA_INIT response.
- A supporting initiator **MAY** send the modified IKE_AUTH request if the notification was included in the IKE_SA_INIT response.
- The initiator **MUST NOT** send the modified IKE_AUTH request if the notification was not present.
- A supporting responder **MUST** process a modified IKE_AUTH request, and **MUST** reply with a modified IKE_AUTH response.

- Such a responder **MUST NOT** reply with a modified IKE_AUTH response if the initiator did not send a modified IKE_AUTH request.

```

      1             2             3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
! Next Payload !C! RESERVED ! Payload Length !
+-+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
! Protocol ID ! SPI Size ! Childless Notify Message Type !
+-+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+

```

CHILDLESS_IKEV2_SUPPORTED Notification

- Code changes and Impact**
 - To make the function for constructing CHILDLESS_IKEV2_SUPPORTED notification payload.
 - Code for generating a modified IKE_AUTH request.
 - A check must be made for the notify payload before generating the IKE_AUTH request.
 - A check at the responder side to be made for modified IKE_AUTH.
 - A further check to permit childless SA depending on parameters like configuration option. (**Note:** This needs to be discussed further.)
 - Code for generating a modified IKE_AUTH response.
 - A supporting responder that has been configured not to support this extension to the protocol **SHOULD** reply with an **INVALID_SYNTAX** Notify payload if the client sends a modified IKE_AUTH request.

Proposed Timeline

May5 - May31 : Community Bonding

- Get to know the mentor.
- Reading through the RFC 5723 and RFC 6023 again to prepare the work split and decide the final use cases for *libreswan*.
- Code walkthrough to understand the current design and take into consideration any possible improvements that can be implemented during the feature implementation.
- Get familiar with how to run test cases using namespaces.
- Propose and discuss the Implementation design with the mentor.

- Finalize the design with the mentor.

June1 - June28

- Try to complete major code changes of RFC 5723 that involves notification payloads, new exchange implementation, config and connection options, etc.
- Testing: Write test cases to manually validate the use cases.
- Analysing the results.
- Resolve the issues as and will be pointed by the mentor in implementation.
- Start working with mentor on design and implementation of next RFC in detail

June29 - July3 : Phase 1 evaluation

July4 - July26

- Presenting the mentor with initial implementation of session resumption feature.
- Collecting reviews from mentor for further improvements in implementation of RFC 5723.
- Think and discuss the type of tests needed with mentor and write their description.
- Start documenting this feature implementation.
- Finalise a full working flow for RFC 6023 and start working on its implementation.

July27 - July31 : Phase 2 evaluation

August1 - August 23

- Wrap up with the implementation of the second RFC.
- Perform manual testing, capture and analyze results.
- Add documentation.

August24 - August31 : Last week

- Adding comments to make code more readable.
- Upload wiki page (if applicable).
- Final review with the mentor.

NOTE: The timeline is subjected to further changes as per the time taken in implementation of RFC 5723.

Time Commitment

My summer vacations are from 9 May to 12 July. I can easily devote 45-50 hours a week until my college reopens and 30-40 hours per week after that. I intend to complete most of the work before my college reopens.

Other than this project, I have no other commitments/vacations planned for the summers.

I shall keep my status posted to all the community members on a weekly basis and maintain transparency in the project.

Post GSoC

I plan to keep contributing to Libreswan after the GSoC period because I believe that there are some tasks or issues which may not be covered during the project period either due to the limited time span or there might be new issues that may come up during the project period accommodating which might be difficult. But I do want to see the task through to the end, so I will keep working on the described implementations and will also help future contributors towards the same.

Furthermore, this will also give me an opportunity to put my skills into practical use and give back to the community which I wholeheartedly want to keep doing through the platform of Libreswan.